

哈尔滨工业大学

编译系统 2024 春

实验二

学院:	未来技术学院
姓名:	郭庆吉
学号:	2021113250
指导教师:	朱庆福

一、实验目的

1. 巩固对词法分析与语法分析的基本功能和原理的认识
2. 能够应用自动机的知识进行词法与语法分析
3. 理解并处理词法分析与语法分析中的异常和错误

二、实验环境

Ubuntu 20.04、GCC 11.4、Flex 2.6.4、Bison 3.8.2

三、实验过程

实验内容

本次实验完成了对于 C—语言的语义分析，对于语义错误的检测。

数据结构

本次实验是承接实验 1 进行的，所以采用的数据结构和实验 1 一样，同为语法分析树。在本次实验中新增的数据结构有两个：变量类型 `Type` 以及字符表中的一个域 `FieldList`。这两个数据结构按照参考书规定的方法进行编写即可，在此不过多赘述。

语义分析

本次实验我们默认 C—文法中词法和语法是不发生错误的。这样我们需要先利用实验 1 的相关内容进行词法分析和语法分析，以便验证词法和语法的正确性，同时我们也能够在这个过程中构造语法树，从而开始我们的语义分析过程。

在实验 1 中，当我们正确构造语法树之后，我们会对于语法树进行遍历，输出每一部分的成分。在本实验中，我们同样需要进行语法树的遍历来实现语义分析。我们在遍历语法书的过程中，依照不同类型的错误原因展开分析，将这种错误对应到语法树上，这样当我们遍历语法树的时候碰到了相同的情况，这就可以作为错误评判的依据。下面展示一些错误的情况的判定方法：

1. 错误类型 1：变量使用时未定义。

```
else if (!strcmp(t->name, "ID")) {
    pItem tp = searchTableItem(table, t->val);
    if (tp == NULL || isStructDef(tp)) {
        char msg[100] = {0};
        sprintf(msg, "Undefined variable \"%s\".", t->val);
        pError(UNDEF_VAR, t->lineNo, msg);
        return NULL;
    } else {
        // good
        return copyType(tp->field->type);
    }
}
```

检查一个名为"ID"的标识符是否已经在表中定义。如果没有定义,或者它是一个结构体定义,它会产生一个错误。否则,它会返回一个与该标识符关联的类型。`searchTableItem(table, t->val)`尝试在表中查找与标识符相关的条目。

如果找不到条目(即 `tp == NULL`),或者找到的条目是一个结构体定义(即 `isStructDef(tp)`),它会生成一个错误消息,指出未定义的变量,然后返回 `NULL`。

如果找到的条目是一个已定义的变量,则会返回与之关联的类型。

2. 错误类型 5、6: 赋值号两边类型不匹配,赋值号左边出现只有右值的表达式。

对应于语法分析的 `EXP → Exp ASSIGNOP EXP` 这一部分。我们声明一个域,当匹配到 `root->child->brother->unit` 为 `ASSIGNOP` 时,我们进行错误检测,首先判断左边是否是 `INT` 或是 `FLOAT`,如果是,则形如 `1=...` 这样的形式,这显然时左边是一个右值表达式的错误,输出错误类型 5。如果左边 `id` 的 `basic` 和右边 `id` 的 `basic` 不一样说明两边表达式类型不匹配,说明不可进行赋值操作,输出错误类型 6。

3. 错误类型 7: 操作数类型不匹配或操作数类型与操作符不匹配。

```
else {
    if (p1 && p2 && (p1->kind == ARRAY || p2->kind == ARRAY)) {
        //报错,数组,结构体运算
        pError(TYPE_MISMATCH_OP, t->lineNo,
            "Type mismatched for operands.");
    } else if (!checkType(p1, p2)) {
        //报错,类型不匹配
        pError(TYPE_MISMATCH_OP, t->lineNo,
            "Type mismatched for operands.");
    } else {
        if (p1 && p2) {
            returnType = copyType(p1);
        }
    }
}
```

这段代码描述了在进行移入-规约操作时,特别是对于表达式的规约,例如 `Exp → Exp PLUS Exp` 或者 `Exp → Exp MINUS Exp` 时,需要检查运算符两侧的表达式类型是否相等。如果两侧表达式的类型不相等,则输出错误类型 7,这与赋值语句中的情况相似,即出现了类型不匹配的错误。

4. 错误类型 12: 数组访问操作符"`[...]`"中出现非整数。

每遇到数组访问语句的时候,对应于 `Exp → Exp LB Exp RB`,程序对字节节点第三个属性值 `root->child->brother->brother`,若不为 `int`,说明出现非整数,输出错误类型 12。

5. 错误类型 10: 对非数组型变量使用数组访问操作符。

对子节点的第三个属性值进行判断,若它的 `kind` 不为数组,则说明该节点对应的是非数组变量,输出错误类型 10。

6. 错误类型 13: 对非结构体类型变量使用"`.`"操作符。

```
// Exp -> Exp DOT ID
else {
    pType p1 = Exp(t);
    pType returnType = NULL;
    if (!p1 || p1->kind != STRUCTURE ||
        !p1->u.structure.structName) {
        //报错,对非结构体使用.运算符
        pError(ILLEGAL_USE_DOT, t->lineNo, "Illegal use of \".\".");
        if (p1) deleteType(p1);
    } else if
```

如果返回的类型为空(!`p1`),或者返回的类型不是结构体类型(`p1->kind != STRUCTURE`),或者返回的类型没有结构体名称(!`p1->u.structure.structName`),那么说明点操作符被用于了一个非结构体类型上。在这种情况下,会输出错误类型为 `ILLEGAL_USE_DOT`,提示点操作符的非法使用,并删除可能存在的类型信息,以避免内存泄漏。当点操作符用于一个非结构体类型时,输出错误类型 13。

编译

我编写了 MakeFile 文件用来进行实验的运行，其中设置了对应语言使用的编译环境以及一些伪目标。

```
8 CFLAGS = -std=c99
9
10 # 编译目标: src目录下的所有.c文件
11 CFILES = $(shell find ./ -name "*.c")
12 OBJS = $(CFILES:.c=.o)
13 LFILE = $(shell find ./ -name "*.l")
14 YFILE = $(shell find ./ -name "*.y")
15 LFC = $(shell find ./ -name "*.l" | sed s/[/]\.l/lex.yy.c/)
16 YFC = $(shell find ./ -name "*.y" | sed s/[/]\.y/syntax.tab.c/)
17 LFO = $(LFC:.c=.o)
18 YFO = $(YFC:.c=.o)
19
20 parser: syntax $(filter-out $(LFO),$(OBJS))
21     $(CC) -o parser $(filter-out $(LFO),$(OBJS)) -lfl
22
23 syntax: lexical syntax-c
24     $(CC) -c $(YFC) -o $(YFO)
25
26 lexical: $(LFILE)
27     $(FLEX) -o $(LFC) $(LFILE)
28
29 syntax-c: $(YFILE)
30     $(BISON) -o $(YFC) -d -v $(YFILE)
31
32 -include $(patsubst %.o, %.d, $(OBJS))
33
34 # 定义的一些伪目标
35 .PHONY: clean test
36 test:
37     ./parser ../Test/test1.cmm
38 clean:
39     rm -f parser lex.yy.c syntax.tab.c syntax.tab.h syntax.output
40     rm -f $(OBJS) $(OBJS:.o=.d)
41     rm -f $(LFC) $(YFC) $(YFC:.c=.h)
42     rm -f *
```

四、实验结果

```
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input1.cmm
Error type 1 at Line 4: Undefined variable "j".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input2.cmm
Error type 2 at Line 4: Undefined function "inc".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input3.cmm
Error type 3 at Line 4: Redefined variable "l".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input4.cmm
Error type 4 at Line 6: Redefined function "func".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input5.cmm
Error type 5 at Line 4: Type mismatched for assignment.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input6.cmm
Error type 6 at Line 4: The left-hand side of an assignment must be a variable.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input7.cmm
Error type 7 at Line 4: Type mismatched for operands.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input8.cmm
Error type 8 at Line 4: Type mismatched for return.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input9.cmm
Error type 9 at Line 8: too many arguments to function "func", except 1 args.
```

```
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input10.cmm
Error type 10 at Line 4: "i" is not an array.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input11.cmm
Error type 11 at Line 4: "i" is not a function.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input12.cmm
Error type 12 at Line 4: "1.5" is not an integer.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input13.cmm
Error type 13 at Line 9: Illegal use of ".".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input14.cmm
Error type 14 at Line 9: NONEXISTFIELD.
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input15.cmm
Error type 15 at Line 4: Redefined field "x".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input16.cmm
Error type 16 at Line 6: Duplicated name "Position".
qqj@gqj-virtual-machine:/mnt/hgfs/ubuntu共享/lab2/Code$ ./parser input17.cmm
Error type 17 at Line 3: Undefined structure "Position".
```

五、实验总结

1. 通过本次实验，我更深刻地理解了语法分析树的构建和作用。从实验 2 结束后回顾实验 1，我意识到语法分析树是理解程序结构和语法的重要工具。通过构建语法分析树，我能够更清晰地理解编程语言的结构和逻辑。

2. 本次实验让我理解了如何从本质上考虑语义分析的错误类型。我学会了将语句拆分成一系列语法单元，并逐个检验是否出现错误以及错误的类型。这种方法帮助我更容易地识别和处理各种语义错误，加深了我对编译器设计中错误处理的理解。

3. 通过完成本次实验中的简单 C 语言语义分析任务，我不仅将课堂学习的知识应用到实践中，还巩固了这些知识。实践是学习的最佳方式，通过编写代码并解决实际问题，我更加深入地理解了编程语言和编译器的工作原理。